

UPPSALA UNIVERSITY



PARALLEL AND DISTRIBUTED PROGRAMMING

1TD070

Malaria Epidemic Simulation

Authors:

Anton VON

NANDELSTADH

June 10, 2026

1 Introduction

The stochastic simulation algorithm (SSA) models the evolution of an epidemic, by categorizing the population into states of susceptible, infected and recovered, stochastically generating a timestep, and changing these states based on certain propensities [5]. Our particular propensities are based on the simulation of a malaria epidemic, although the algorithm generalizes beyond this.

The aim of this project is to parallelize the SSA algorithm using a Monte Carlo approach, where we run multiple independent iterations of the SSA algorithm, and then average over these to gather interesting statistics. In our case, this will be the population of susceptible humans at the end of the simulation.[7]

The original account of the SSA algorithm was given in [5] and a more detailed derivation and justification can be found in [3]. Faster approximations of the SSA algorithm has been considered, such as tau leaping, as in [4]. There are other methods of parallelizing the SSA algorithm, but these are more complicated and offer some tradeoffs regarding accuracy and are limited by the conditions under which they can be applied [8]. Many more parallel event simulation algorithms and a more in-depth literature overview can be found in [2]. More about stochastic simulations of epidemics can be found in [1].

2 Algorithms

Algorithm 1 The MC algorithm

- 1: Choose the number of MC experiments N
 - 2: **for** $i = 1, 2, \dots, N$ **do**
 - 3: Perform one MC experiment and store the result in a suitable vector
 - 4: **end for**
 - 5: Compute some mean value or another quantity, summarizing the results.
-

Algorithm 2 Gillespie's direct method (SSA)

- 1: Set a final simulation time T , current time $t = 0$, initial state $\mathbf{x} = \mathbf{x}_0$
- 2: **while** $t < T$ **do**
- 3: Compute $w = \text{prop}(\mathbf{x})$
- 4: Compute $a_0 = \sum_{i=1}^R w^{(i)}$
- 5: Generate two uniform random numbers u_1, u_2 between 0 and 1
- 6: Set $\tau = -\ln(u_1)/a_0$
- 7: Find r such that

$$\sum_{k=1}^{r-1} w_k < a_0 u_2 \leq \sum_{k=1}^r w_k$$

- 8: Update the state vector $\mathbf{x} = \mathbf{x} + P(r, :)$
 - 9: Update time $t = t + \tau$
 - 10: **end while**
-

We use algorithm 1 in conjunction with 2. In particular, we use the Monte Carlo algorithm to simulate the amount of susceptible humans at $t = 100$. We estimate the probability distribution by running the SSA algorithm multiple times, and then plotting a histogram of the number of susceptible humans by the end of each simulation.

We initialized the algorithm with $x_0 = [900, 900, 30, 330, 50, 270, 20]$ and used $T = 100$.

The Monte-Carlo algorithm is very simple and just relies on the Law of Large Numbers and the Central Limit Theorem. These state that under appropriate conditions, the mean will converge, and that the distribution it converges to is the normal distribution. [6] The Monte Carlo algorithm is simply using this to estimate the distribution, in our case of susceptible humans.

Clearly, each individual simulation using the SSA epidemic is completely serial, since each time state depends on the previous. Moreover, the Monte Carlo algorithm assumes that these simulations are completely independent, and hence it is perfectly parallel. Thus, we can run simulations independently over separate processors, and once they are all finished, we can compute any required statistic.

Our implementation of the SSA algorithm is exactly like the pseudocode and since it was completely serial, it was not the focus of our optimization

attempts. The optimizations we did here were to declare constant values as constant, and precomputing values outside of the innermost loop, whenever possible. Rather, we focused on optimizing the computation of our histogram, and the computation of the average time taken per interval of the Gillespie algorithm. The local computation of average times is built into our main loop.

Listing 1: Main loop

```

1  for (int i = 0; i < n; i++) {
2      X[i][0] = 900;
3      X[i][1] = 900;
4      X[i][2] = 30;
5      X[i][3] = 330;
6      X[i][4] = 50;
7      X[i][5] = 270;
8      X[i][6] = 20;
9
10     t = 0;
11     k = 0;
12     double passed_time[4] = {0};
13     // Main loop for each individual process
14     double simulation_start = MPI_Wtime();
15     while (t < T) {
16         double t_prev = t;
17         t += gillespie(X[i], w, P);
18         while (k < 4 && t_prev < checkpoints[k] && t
19                 >= checkpoints[k]) {
20             passed_time[k] = MPI_Wtime() -
21                             simulation_start;
22             k++;
23         }
24     }
25     times[0] += passed_time[0];
26     times[1] += passed_time[1] - passed_time[0];
27     times[2] += passed_time[2] - passed_time[1];
28     times[3] += passed_time[3] - passed_time[2];
29 }

```

At the end of our inner for loop, we have the times per interval on each process, but we still need to reduce this for a computation of the global mean,

which is done on the root.

Listing 2: Reduction of times.

```
1 // Reducing interval times
2 double all_times[n_procs][4];
3 double global_times[4] = {0};
4
5 MPI_Win win_times;
6
7 MPI_Win_create(all_times, myid == 0 ? n_procs * 4 *
8     sizeof(double) : 0, sizeof(double), MPI_INFO_NULL
9     , MPI_COMM_WORLD, &win_times);
10
11 MPI_Win_fence(0, win_times);
12 MPI_Put(times, 4, MPI_DOUBLE, 0, myid * 4, 4,
13     MPI_DOUBLE, win_times);
14 MPI_Win_fence(0, win_times);
15
16 // Reducing global times
17 double maxtime;
18 MPI_Reduce(&time, &maxtime, 1, MPI_DOUBLE, MPI_MAX,
19     0, MPI_COMM_WORLD);
```

We used one-sided communication for the reduction, but did not find a significant improvement over regular MPI reduction. This is likely because we compute the sum of all times on each processor, and only have to exchange values with the root node once. This means that communication overhead is not very significant, since we only need to exchange $4p$ doubles in p communications. After communicating the sums from each processors to the node using a window, we still need to manually compute the averages on the root. Note that we need to synchronize before and after the Put function is called. The first synchronize is to make sure that we have opened the window before trying to access it, and the second is to make sure that all processes have Put their data before we try to modify it on the root.

We used regular MPI reduction for the global time, since this made the computation cleaner. Moreover, we are not really timing the computation of the time anyway.

Other than the serial simulations of the SSA algorithm, our timed section also includes

Listing 3: Histogram computation.

```

1 void histogram(int n, int X[n][7], int hist[NUM_BUCKETS
   ], int* gmax, int* gmin) {
2     int max, min;
3     max = X[0][0];
4     min = X[0][0];
5     for (int i = 0; i < n; i++) {
6         if (X[i][0] < min) min = X[i][0];
7         if (X[i][0] > max) max = X[i][0];
8     }
9
10    int n_procs, myid;
11    MPI_Comm_size(MPI_COMM_WORLD, &n_procs);
12    MPI_Comm_rank(MPI_COMM_WORLD, &myid);
13
14    MPI_Allreduce(&max, gmax, 1, MPI_INT, MPI_MAX,
15                 MPI_COMM_WORLD);
16    MPI_Allreduce(&min, gmin, 1, MPI_INT, MPI_MIN,
17                 MPI_COMM_WORLD);
18
19    double bucket_width = ((double)(*gmax - *gmin)) /
20                          NUM_BUCKETS;
21    if (bucket_width == 0) {
22        printf("gmin==gmax, something went wrong\n");
23        return;
24    }
25
26    int buckets[NUM_BUCKETS] = {0};
27
28    // Fill histogram
29    for (size_t i = 0; i < n; i++) {
30        int index = (int)((X[i][0] - *gmin) /
31                          bucket_width);
32
33        // Ensure max value goes into last bucket
34        if (index == NUM_BUCKETS)
35            index = NUM_BUCKETS - 1;
36
37        buckets[index]++;
38    }
39 }

```

```

35 MPI_Reduce(&buckets, hist, NUM_BUCKETS, MPI_INT,
36           MPI_SUM, 0, MPI_COMM_WORLD);
37 }

```

Here we first compute local minima and maxima. Then we reduce these to global minima and maxima on the root, before broadcasting it back to all the nodes. Having broadcast the information, each node can locally compute the buckets and fill them, before the final information is reduced back to the root, which can then print or plot the data in the main function. The idea here is to keep as much of the computation as possible distributed, so as to minimize the computation done single-handedly by the root, and thus improve load balance.

An alternative would be to use one-sided communication and send information continuously to the root as the simulations happen. However, this would hugely increase the number of communications, which is the most expensive part of MPI programmes. Hence we chose to do as much local computation as possible. We were not really able to use parallel IO either, since we had to sum up the values in each bucket, which meant that any parallel IO could have caused race conditions.

3 Experiments and Results

The experiments were done on the Pelle cluster of Uppsala University's HPC. The experiments used two nodes with up to 48 processors, in order to stress communication and load balance. We used the OpenMPI flag `--map-by ppr:p/2:node`, where `p` is the total number of processors, to evenly allocate the processors over the two nodes. The weak scaling experiments were done using 10^5 simulations per processor, while the strong scaling experiments were done using slightly over 10^6 simulations over all the processors. (Note that we in fact ran 1000032 simulations for the strong scaling experiment, so that the number of simulations would be divisible by 48 as well, since our programme assumes that the number of simulations is divisible by the number of processors.)

Our timed section included the main loop of SSA simulations, as well as the gathering of data and computation of our histogram. It does not include the reduction of timing information, printing of information or the actual

plotting the histogram, since these are either not our main interest or highly system dependent.

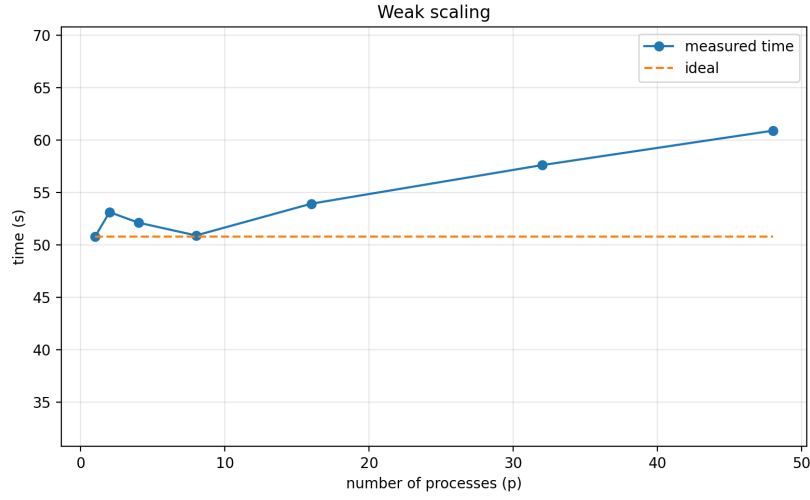


Figure 1: Weak scaling

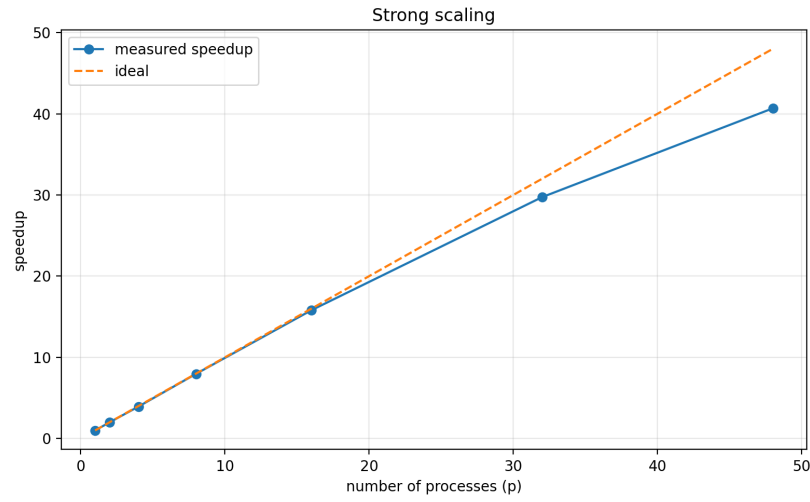


Figure 2: Strong scaling

Tab. 2 contains two subtables. On the left is the average time for each processor, taken per interval 0 – 25, 25 – 50, 50 – 75 and 75 – 100 for the

(a) Weak scaling			(b) Strong scaling		
$t(s)$	n	p	$t(s)$	n	p
50.784771	100000	1	508.121106	1000032	1
53.119704	200000	2	254.687094	1000032	2
52.117895	400000	4	129.253537	1000032	4
50.897940	800000	8	63.766585	1000032	8
53.925012	1600000	16	32.145060	1000032	16
57.600540	3200000	32	17.088205	1000032	32
60.881411	4800000	48	12.492881	1000032	48

Table 1: Weak and strong scaling results

SSA algorithm. The subtable to the right shows the speedup relative to the number of processors. Both of the tables use the same data as our strong scaling results.

(a) Interval times					(b) Speedup	
p	t_{25}	t_{50}	t_{75}	t_{100}	p	speedup
0	0.000253	0.000097	0.000079	0.000080	1	1.000
1	0.000251	0.000097	0.000079	0.000080	2	1.995
2	0.000251	0.000097	0.000079	0.000080	4	3.931
3	0.000251	0.000097	0.000079	0.000080	8	7.968
4	0.000254	0.000097	0.000079	0.000080	16	15.807
5	0.000252	0.000097	0.000079	0.000080	32	29.735
6	0.000253	0.000097	0.000079	0.000080	48	40.673
7	0.000251	0.000097	0.000079	0.000080		

Table 2: Interval times and speedup

The histograms from 10^3 and 10^6 simulations are given by Fig. 3 These illustrate convergence to the normal distribution as the number of simulations grow.

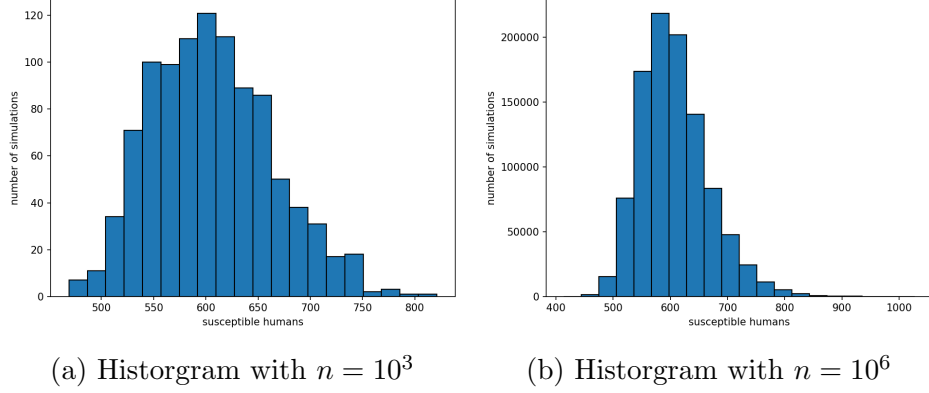


Figure 3: Histograms

4 Discussion

We got very good scaling for our results, which is not very surprising, since the communication overhead in our algorithm is small. Since we care about means over processes, we can compute local sums on each process, which then only communicate after all of the simulations have finished. This means that we need no global barriers during the simulations, and the simulations are completely parallel. Moreover, the communication scales linearly with the number of processes.

For the average interval times, we got two rather surprising results. The first is that the time per interval seems to decrease with time. The time steps in the SSA algorithm are given by $\tau = -\frac{\log(u_1)}{a_0}$. This seems to indicate that a_0 drops with time, making the time steps longer. The second result is that the time per interval increases slightly when the number of processes get close to saturating the node. Since the simulations are completely independent, this should not theoretically happen. However, we believe that this is system dependent and due to the node being close to saturation. Unfortunately we were limited by the project's computational budget, since we believe that using fewer processes per node and more nodes would have mitigated this problem.

On 48 nodes, we got an improvement by a factor of 40 for our strong scaling experiments. This is very good. Up to 16 processes, the speedup is almost linear. However, for larger process counts, the time taken by the sequential part of the algorithm becomes relatively smaller (Amdahl's law),

and the overhead larger.

Our weak scaling is not significantly better, but here we must note that due to computation constraints, we used fewer simulations for our computation of weak scaling. We found that 10^5 simulations per processor took about 50s on one processor and 60s on 48 processors, which is still very respectable. Some of this could also be due to the problem of node saturation discussed above, which would give even better theoretical performance for our algorithm. Nevertheless, some of the slowdown is also likely due to overhead and Gustafson’s law.

In general, it seems like the Monte Carlo algorithm is very well suited for parallelization. This is because each of the individual trials will be completely independent from each other, and communication only has to occur once the processors are finished with their local trials. As long as one is smart about the reduction of data, the communication overhead will be minimal.

The largest possible improvement in our case would not be to the Monte Carlo algorithm, but rather to the serial Gillespie algorithm itself. There have been previous attempts at this, and they might be successful for the problem of malaria pandemic simulation [8]. One could perhaps apply a shared-memory parallelization on each node of a cluster, and then the Monte Carlo algorithm across nodes.

5 AI Declaration

Github Copilot with GPT 5.3 Codex was used to aid with debugging and understanding compiler errors. The most common prompt was ”why does this segfault”. Copilot was also used to generate all of the python code for plotting. Copilot also generated the latex typesetting of tables, based on the data from our experiments.

6 References

References

- [1] Linda J. S. Allen. *An Introduction to Stochastic Epidemic Models*. Springer, 2008.

- [2] Richard M. Fujimoto. “Parallel Discrete Event Simulation”. In: *Communications of the ACM* 33.10 (1990), pp. 30–53.
- [3] Daniel T. Gillespie. “A General Method for Numerically Simulating the Stochastic Time Evolution of Coupled Chemical Reactions”. In: *Journal of Computational Physics* 22.4 (1976), pp. 403–434. DOI: 10.1016/0021-9991(76)90041-3.
- [4] Daniel T. Gillespie. “Approximate accelerated stochastic simulation of chemically reacting systems”. In: *The Journal of Chemical Physics* 115.4 (2001), pp. 1716–1733. DOI: 10.1063/1.1378322.
- [5] Daniel T. Gillespie. “Exact stochastic simulation of coupled chemical reactions”. In: *The Journal of Physical Chemistry* 81.25 (Dec. 1977), pp. 2340–2361. DOI: <https://doi.org/10.1021/j100540a008>.
- [6] Olav Kallenberg. *Foundations of Modern Probability*. Springer, Jan. 2002. ISBN: 9780387953137.
- [7] Davoud Mirzaei. *Stochastic Simulation and Monte Carlo Method*. 2025. arXiv: 2501.00997 [math.NA]. URL: <https://arxiv.org/abs/2501.00997>.
- [8] Azmi Mohamed Ridwan, Arun Krishnan, and Pawan Dhar. “A Parallel Implementation of Gillespie’s Direct Method”. In: (2004). Ed. by Marian Bubak et al., pp. 284–291.